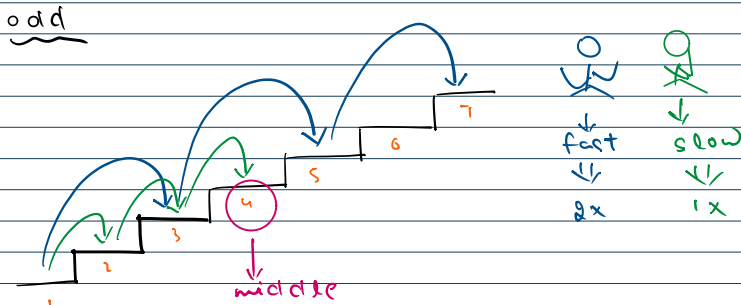
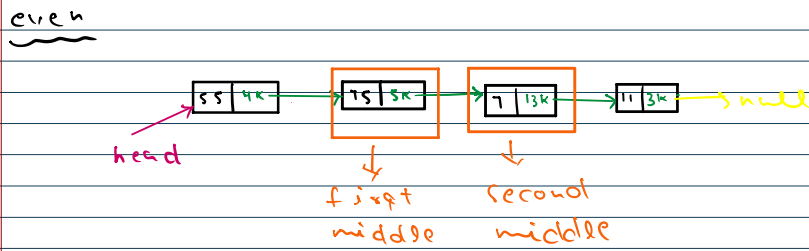
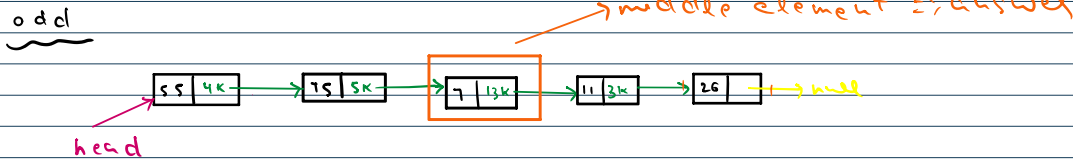
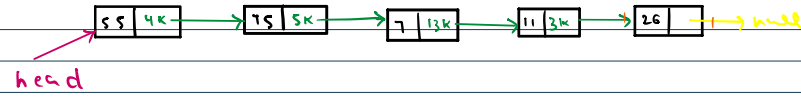
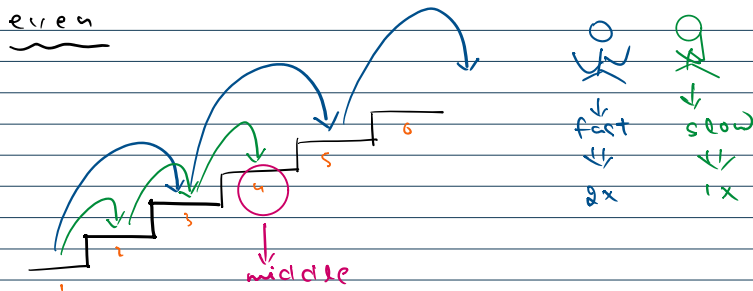


middle of the Linked List

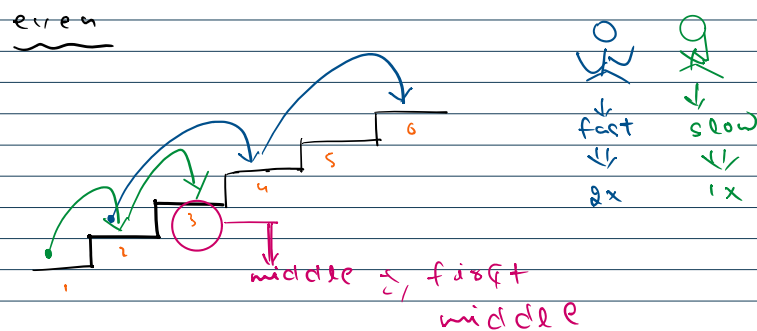
length $\begin{cases} \rightarrow \text{even} \\ \rightarrow \text{odd} \end{cases}$



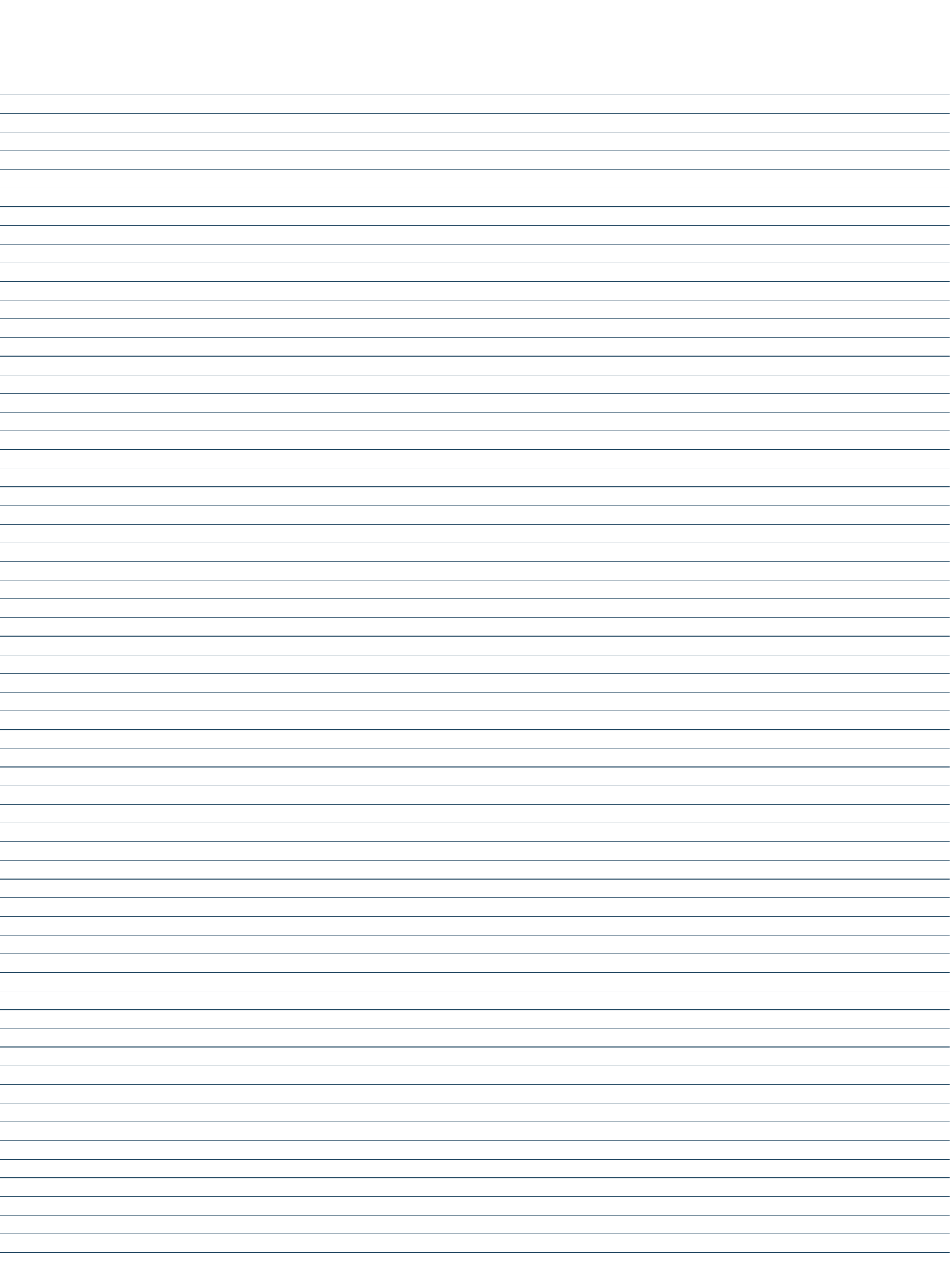
fast == null
slow $\rightarrow$ ans



fast.next == null
slow $\rightarrow$ ans



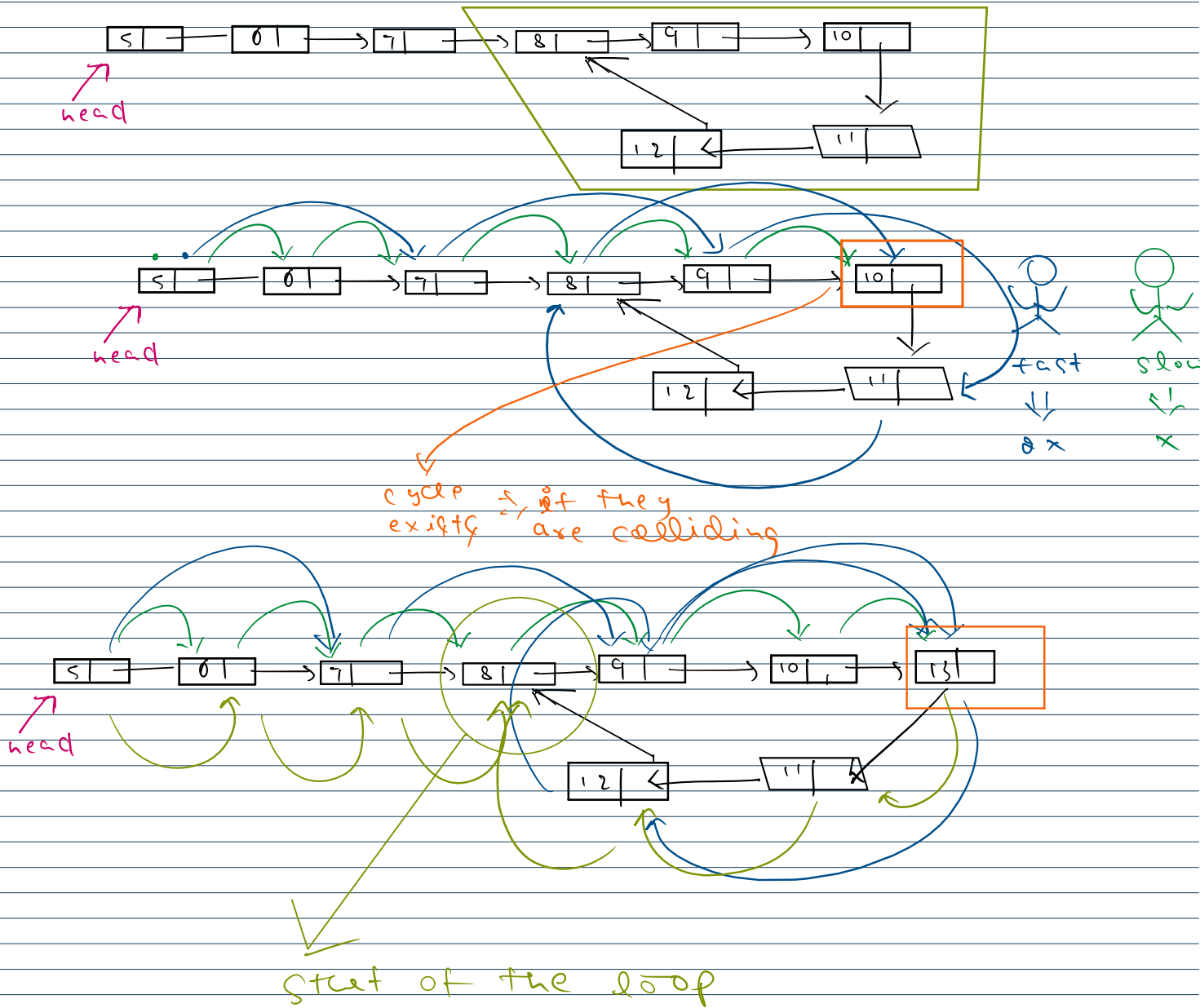
middle is first middle



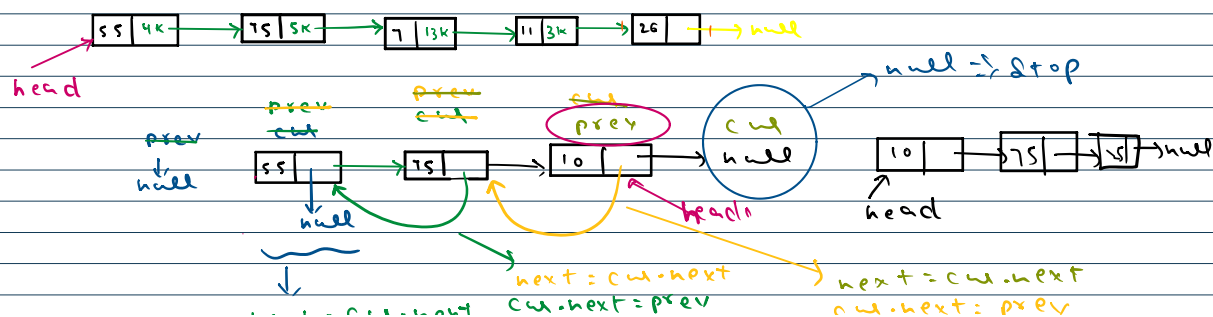
Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter.**

Return `true` if there is a cycle in the linked list. Otherwise, return `false`.



Reverse Linked List



Blank lined paper for writing.

~~next = cur.next~~
~~cur.next = prev~~
~~prev = cur~~
~~cur = next~~

~~next = cur.next~~
~~cur.next = prev~~
~~prev = cur~~
~~cur = next~~

~~next = cur.next~~
~~cur.next = prev~~
~~prev = cur~~
~~cur = next~~

```

class Solution {
public:
    ListNode reverseList(ListNode head) {
        ListNode cur = head;
        ListNode prev = null;
        ListNode next = null;

        while (cur != null) {
            next = cur.next;
            cur.next = prev;
            prev = cur;
            cur = next;
        }

        return prev;
    }
}

```

